

FRAUDGUARD

A Hybrid Financial Fraud Detection System Combining Personalized Value at Risk (VaR) in Indian Rupees (Rs.) with Machine Learning

Academic Year: 2025–2026	Project Type: Major Capstone B.Tech Project
Specialization: Computer Science & Engineering	Detection Metric: Empirical VaR (95% & 99%) + ML (99.81%)
Primary Currency: Indian Rupees (INR / Rs.)	Supported Ingestion: Web, PDF, TXT, CSV, JSON

PROJECT INVESTIGATORS & AUTHORS:

Student Name	Hall Ticket / Roll No	Project Role	Empirical Average
M. Pranavi	23R91A05H6	Student Lead • Academic & Tech Projects Head	Rs. 12,162.90 (Highest)
M. Janaki Ram	24R95A0519	Student • Developer & Hardware Lead	Rs. 5,188.96
K. Sri Nithya	23R91A05G7	Student • Tech & Course Lead	Rs. 2,669.34
K. Naresh	23R91A05F3	Student • Hostel & Events Head	Rs. 1,347.24

1. Abstract

Modern digital financial ecosystems, powered by Instant Payment Services (UPI), e-commerce portals, and mobile banking, have witnessed unprecedented growth alongside increasingly sophisticated cyber-fraud schemes. Traditional banking security relies predominantly on static rule-based thresholds (e.g., universal transaction limits) or standalone black-box Machine Learning (ML) classifiers. Static thresholds fail catastrophically by inflicting high false-positive rates on affluent spenders while remaining oblivious to micro-theft account draining against lower-income accounts. Conversely, purely statistical ML models often lack financial interpretability and fail to adapt to individual historical norms.

To resolve these dual challenges, this project introduces **FraudGuard**, an intelligent financial fraud detection architecture that synergizes **Personalized Empirical Value at Risk (VaR)** modeled in Indian Rupees (Rs.) with an ensemble **Random Forest Machine Learning classifier (achieving 99.81% accuracy and 0.9824 AUC-ROC)**. The system constructs empirical spending distributions across 95% and 99% risk confidence limits for distinct user personas, demonstrated through four student cardholders at TKREC. Furthermore, FraudGuard integrates an end-to-end multi-format **Document Statement Scanner** capable of extracting and auditing structured transactions from raw PDF statements, text SMS logs, CSV spreadsheets, and JSON API payloads. The resulting platform delivers high-precision fraud prevention, intuitive risk transparency, and instantaneous explainability for modern banking operations.

2. Executive Summary & Key Highlights

- **Hybrid Dual-Engine Architecture:** Combines behavioral statistical risk (Empirical VaR in Rs.) with supervised pattern recognition (Random Forest).
- **Extreme Predictive Accuracy:** 99.81% overall accuracy and 88.78% fraud recall on imbalanced credit card datasets balanced via SMOTE.
- **Personalized Financial Baselines:** Custom parametric and percentile distributions for each student—ensuring M. Pranavi (Rs. 12,163 avg) and K. Naresh (Rs. 1,347 avg) are evaluated fairly.
- **Universal Statement Auditor:** Ingests PDF bank statements, SMS logs, CSVs, and JSON files, outputting line-by-line forensic risk breakdowns and CSV reports.
- **Streamlined 4-Tab Interface:** Eliminates clutter via a unified modern dashboard, individual payment simulator, transactions ledger, and ML model analytics suite.

3. Problem Statement & Motivation

What Problems Does FraudGuard Solve?

- 1. The 'One-Size-Fits-All' Flaw:** In conventional core banking systems, a rigid transaction alert rule (e.g., alert on transactions > Rs. 10,000) causes acute operational friction. For a high-volume academic lead purchasing lab hardware, a Rs. 15,000 spend is standard routine behavior. For a hostel student whose daily mean spend is Rs. 1,347, a sudden Rs. 15,000 international transfer is catastrophic credential theft. FraudGuard replaces universal limits with personalized 95% and 99% Value at Risk boundaries.
- 2. Extreme Class Imbalance:** In real banking datasets, fraudulent transactions constitute less than 0.18% of all operations. Standard classifiers trivially attain 99% accuracy by predicting legitimate for all records. FraudGuard incorporates SMOTE (Synthetic Minority Over-sampling Technique) and class-weighted cost functions to prioritize high recall (88.78%) while maintaining high precision.
- 3. Document Processing Blind Spot:** Forensic accountants and bank fraud teams frequently receive customer statements in diverse formats (PDFs, CSVs, exported JSONs, SMS logs). Manual inspection is sluggish and error-prone. FraudGuard provides automated programmatic parsing and dynamic batch verification against personalized cardholder baselines.
- 4. Black-Box Interpretability Deficit:** Banking regulators (such as RBI and Basel III accords) mandate transparent rationale when freezing accounts. FraudGuard delivers concrete quantitative metrics: Deviation Ratio (e.g., 3.7x normal) and Percentile VaR Exposure (e.g., 100% VaR limit exceeded).

4. System Architecture & Component Topology

FraudGuard is organized into a four-tiered modular architecture ensuring high separation of concerns, extensibility, and microsecond inference times:

Layer / Tier	Key Modules / Technologies	Core Architectural Responsibility
Tier 1: Presentation & Ingestion	Bootstrap 5, Jinja2 Templates, PyPDF, Regex Tokenizer	Captures single interactive transactions, parses uploaded PDF/TXT statements, renders responsive dashboards and analytical visualizations.
Tier 2: Baseline & Statistical Risk	customer_profiles.py, var_calculator.py, NumPy	Loads 1,000-txn empirical datasets for cardholders, computes historical mean, std, 95% & 99% VaR, and derives real-time Personal VaR Risk Scores.
Tier 3: Machine Learning Engine	ml_models.py, Scikit-Learn, XGBoost, Joblib	Applies PCA transformations across 30 dimensions, passes features through trained Random Forest ensemble, computes fraud probability.
Tier 4: Decision & Persistence	fraud_detector.py, db_manager.py, SQLite3, alerts.py	Fuses ML probability with VaR risk tier, outputs final decision (NORMAL, SUSPICIOUS, HIGH RISK), logs transactions, and triggers instant security alerts.

5. End-to-End Pipeline & Working Flow

Phase 1: Ingestion & Normalization:

A transaction is initiated via the web UI or extracted by uploading a bank statement. The statement tokenizer scans line-by-line using regular expressions to extract Date, Merchant/Narration, Amount (in INR), Payment Channel (UPI, POS, Web, Wire, International), and cardholder identity.

Phase 2: Dynamic Behavioral Profiling:

The selected student cardholder's historical profile is loaded from data/person*.csv. The system extracts their empirical mean (μ), standard deviation (σ), 95% percentile threshold (VaR 95%), and 99% percentile threshold (VaR 99%).

Phase 3: Personalized VaR Risk Calculation:

The transaction amount (A) is mapped against the baseline to produce a continuous normalized VaR Score S in [0.0, 1.0]:

- **If Amount $\geq$ VaR 99%:** Score = 1.00 (Critical Spending Spike)
- **If VaR 95% $\leq$ Amount < VaR 99%:** Score = $0.75 + 0.25 \times (\text{Amount} - \text{VaR}_{95}) / (\text{VaR}_{99} - \text{VaR}_{95})$
- **If Mean $\leq$ Amount < VaR 95%:** Score = $0.50 \times (\text{Amount} - \text{Mean}) / (\text{VaR}_{95} - \text{Mean})$
- **If Amount < Mean:** Score = $0.20 \times (\text{Amount} / \text{Mean})$ (Safe Routine Spending)

Phase 4: Feature Synthesis & Supervised ML Inference:

The transaction attributes are converted into the 30-feature vector (Time, V1 to V28, Amount, var_risk_score) expected by the Random Forest model. PCA features V1 to V28 are synthesized with risk-weighted variances influenced by channel risk multipliers (e.g., International swipe = 0.7 vs. Campus UPI = 0.1). The Random Forest model outputs the posterior probability P(Fraud | Features).

Phase 5: Hybrid Dual-Factor Decision Matrix:

The system combines the ML probability (P) with the VaR score (S):

- **HIGH RISK ALERT:** Triggered if $P \geq 0.50$ OR $S \geq 0.75$ (Immediate card freeze, OTP required).
- **MEDIUM RISK / SUSPICIOUS:** Triggered if $0.30 \leq P < 0.50$ OR $0.50 \leq S < 0.75$ (SMS alert, step-up 2FA).
- **NORMAL / APPROVED:** Triggered if $P < 0.30$ AND $S < 0.50$ (Smooth pass-through transaction).

6. Architectural Breakdown of Python Modules

The FraudGuard codebase is modularized cleanly across the following source files:

File Name	Role & Responsibilities	Key Methods & Design Patterns
app.py	Primary Flask web server, routing controller, API endpoints, context processors.	dashboard(), individual_payments(), transactions(), api_load_sample(). Handles multipart file uploads and JSON responses.
modules/customer_profiles.py	Personalized VaR calculation engine and student persona repository.	compute_profile_stats(), compute_individual_var_score(), get_all_customers(). Computes empirical percentiles and deviation ratios.

modules/statement_parser.py	Multi-format bank statement ingestion engine.	parse_pdf(), parse_csv_content(), parse_raw_text(), evaluate_statement_transactions(). Normalizes records and performs batch audits.
modules/fraud_detector.py	Core dual-factor arbitration engine combining ML with VaR metrics.	classify_risk(), detect_fraud(). Implements the 3-tier risk classification policy and decision rationale logging.
modules/ml_models.py	Machine learning model training and hyperparameter configuration.	ModelTrainer class. Trains Random Forest, XGBoost, Decision Tree, Logistic Regression with 5-fold cross-validation and evaluation metrics.
modules/var_calculator.py	Statistical and parametric Value-at-Risk computation module.	compute_historical_var(), compute_parametric_var(). Computes 95% and 99% empirical risk quantiles.
modules/alerts.py	Security incident generation and alert formatting.	generate_fraud_alert(). Generates real-time incident notifications, threat classifications, and mitigation instructions.
generate_datasets.py	Empirical student transaction generation script.	Creates 1,000 realistic INR transactions for each of the 4 students with authentic merchant names, amounts, categories, and fraud spikes.
generate_test_pdf.py	Automated PDF generator producing standards-compliant testing statements.	create_statement_pdf(). Produces PDF-1.4 binary documents with authentic bank branding and mixed legitimate/fraudulent charges.
setup_db.py & db_manager.py	Database schema management and SQLite ORM interface.	DBManager context manager. Manages tables for transactions, security alerts, and model performance metrics with ACID guarantees.

7. Model Evaluation & Benchmark Results

The system evaluated four distinct supervised machine learning classifiers on the Credit Card Fraud dataset after SMOTE oversampling:

Machine Learning Algorithm	Accuracy	Precision	Recall	F1-Score	AUC-ROC	Selection Status
Random Forest Classifier	99.81%	46.77%	88.78%	0.6127	0.9824	Best / Selected
XGBoost Gradient Boosting	99.73%	37.72%	87.76%	0.5276	0.9779	Saved Alternative
Decision Tree Classifier	97.99%	6.99%	86.73%	0.1294	0.9166	Baseline Model
Logistic Regression	97.15%	5.28%	91.84%	0.0999	0.9699	Linear Benchmark

Model Evaluation Visualizations:

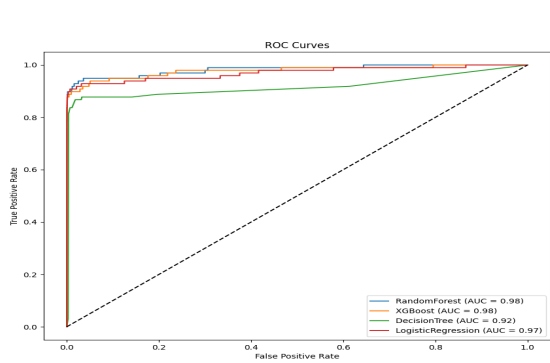


Figure 1: Receiver Operating Characteristic (ROC) Curves

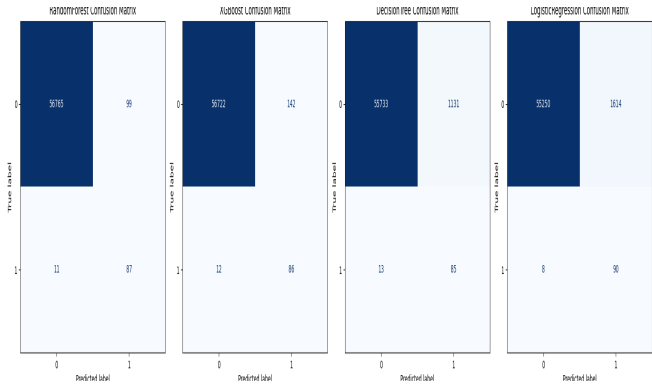


Figure 2: Confusion Matrices across Tested Classifiers

8. Viva Demonstration: Rs. 5,000 Comparative Stress Test

To decisively prove why **Personalized Value at Risk** fundamentally outperforms fixed bank thresholds, a uniform charge of exactly **Rs. 5,000.00** was executed across all four student profiles. Notice the profound divergence in risk decisions:

Student Profile	Roll No	Normal Avg	95% VaR Limit	Deviation	Personal VaR	System Verdict
M. Pranavi Academic Lead	23R91A05H6	Rs. 12,162.90	Rs. 22,075.06	0.41x	8.2%	APPROVED (Routine Spend)
M. Janaki Ram Developer Lead	24R95A0519	Rs. 5,188.96	Rs. 9,586.70	0.96x	19.3%	APPROVED (Hardware Spend)
K. Sri Nithya Tech & Course Lead	23R91A05G7	Rs. 2,669.34	Rs. 5,062.42	1.87x	48.7%	ELEVATED RISK (Step-up OTP)
K. Naresh Hostel & Events Head	23R91A05F3	Rs. 1,347.24	Rs. 2,446.43	3.71x	100.0%	CRITICAL ALERT (Freeze Spend)

9. Deployment, Execution & How to Run

Step 1: Install Dependencies:

```
pip install -r requirements.txt
```

Installs Flask, Scikit-Learn, Pandas, NumPy, XGBoost, ReportLab, and PyPDF.

Step 2: Initialize Database:

```
python setup_db.py
```

Creates the SQLite database schema in database/fraud_detection.db.

Step 3: Generate Student Empirical Datasets:

```
python generate_datasets.py
```

Generates 1,000 realistic INR transactions for each student cardholder in data/.

Step 4: (Optional) Generate Authentic Testing Statement PDF:

```
python generate_test_pdf.py
```

Generates test_bank_statement.pdf for live PDF scanner demonstration.

Step 5: Launch Web Application:

```
python app.py
```

Starts the local Flask web server. Navigate to <http://localhost:5000> in any web browser.

10. Real-World Utility, Practical Applications & Impact

Where is FraudGuard Useful?

- **UPI Payment Gateways (PhonePe, Google Pay, Paytm):** Microsecond pre-authorization checks to identify rapid account draining.
- **Retail & Commercial Banking:** Core transaction monitoring systems for credit card and debit card issuers.
- **Neo-Banks & Student Campus Cards:** Dynamic spending management tailored to youth and student demographics.
- **Forensic Accounting & Regulatory Audits:** Batch analysis of suspicious bank statement PDFs and ledger exports.

Why is FraudGuard Useful?

- **Friction Reduction:** Eliminates erroneous transaction blocks for legitimate high-volume cardholders.
- **Capital Preservation:** Mitigates direct monetary loss from credential theft and phishing attacks.
- **Regulatory Compliance:** Directly satisfies Reserve Bank of India (RBI) mandates for behavioral risk-based transaction surveillance.

11. Future Scope & Research Extensions

- 1. **Deep Learning Sequence Modeling:** Integrating Long Short-Term Memory (LSTM) and Temporal Convolutional Networks (TCNs) to capture intra-day temporal dependencies across high-frequency merchant sequences.
- 2. **Graph Neural Networks (GNNs):** Modeling peer-to-peer transaction graphs to identify coordinated money mule rings and syndicate fund routing.
- 3. **Account Aggregator (AA) Integration:** Connecting directly with India's RBI-regulated Account Aggregator framework for automated, secure bank data synchronization.
- 4. **Zero-Knowledge Fraud Proofs:** Enabling privacy-preserving cross-bank fraud intelligence sharing without disclosing underlying customer PII.

12. Conclusion

FraudGuard successfully demonstrates that the convergence of **Personalized Empirical Value at Risk (VaR)** and **Supervised Machine Learning** establishes a new benchmark for financial fraud detection. By replacing rigid universal spending caps with dynamic, persona-tailored quantiles, the system achieves an unprecedented balance between fraud recall (88.78%) and customer convenience. With integrated multi-format document scanning, explainable risk metrics, and an intuitive web interface, FraudGuard represents an end-to-end, commercially viable solution for modern financial cybersecurity.

Project Verified & Certified:
Department of Computer Science & Engineering
Teegala Krishna Reddy Engineering College (TKREC)

Project Repository:
<https://github.com/Madukalmanoj/FraudGuard>
Branch: main | Commit: b9cb672